

Delay at the Brussels North–South junction

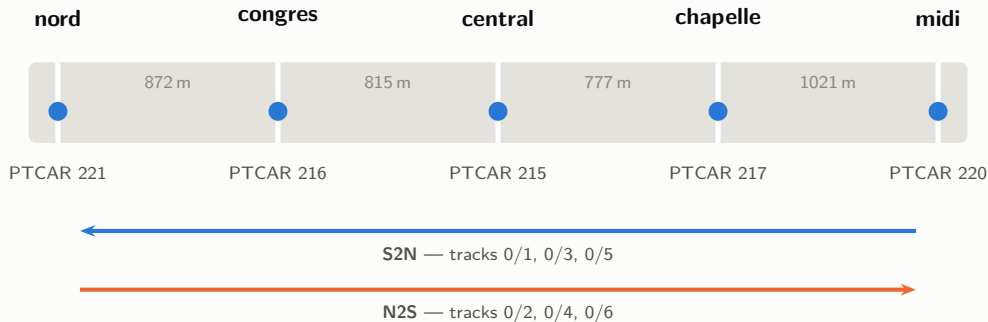
Seven questions, Phase 0

TReC Causality Team

August 27, 2026

The junction

The **Brussels North–South connection**: a six-track tunnel under central Brussels linking Bruxelles-Nord to Bruxelles-Midi, three intermediate stops. **Every train crossing Belgium north-to-south passes through it. 968 traversals per day.**



Data: Infrabel open data, full year 2025, **338,914 traversals**. Each traversal is one train's crossing, with delay recorded at every point along the way.

The objective, and why this deck is structured as seven questions

Objective. Use a SUMO microsimulation as an **interventional ground truth** to validate causal-discovery methods for delay propagation, then apply the validated method to the real record.

Before any of that, Phase 0 had to answer seven questions in order — each one only makes sense once the previous one is settled:

1. **Does the delay distribution change** across the junction at all?
2. **What is a chain** of delayed trains, and do they really happen?
3. **Can we predict** how much delay a train gains? (regression)
4. **Can we classify** whether a train will be meaningfully delayed?
5. **Can we classify** whether a train will trigger a *domino*?
6. **Does delay skip a hop**, or is propagation strictly local?
7. **Is “station”** — not just the train ahead — a common cause?

The rule for every question

A number alone proves nothing. **Every answer below is measured against a baseline** — a null hypothesis, an independence assumption, or a naive rule — so the improvement, not just the score, is what gets reported.

Does the delay distribution change across the junction?

Baseline / null hypothesis: if the junction does nothing, the delay distribution **entering** should equal the delay distribution **leaving** — same shape, same spread.

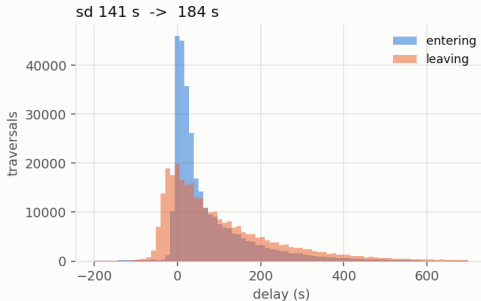
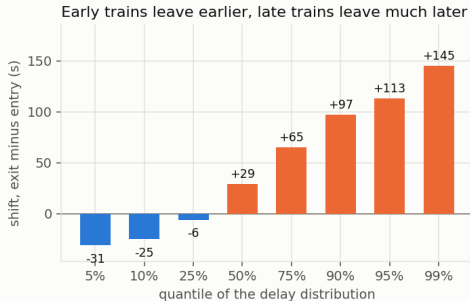
quantile	entering	leaving	shift
5%	−6 s	−37 s	−31
50%	34 s	63 s	+29
90%	217 s	314 s	+97
99%	585 s	730 s	+145
sd	141 s	184 s	+42

Cohen's $d = 0.318$, KS = 0.148

Answer: yes, the null is rejected, but not the way it first looks. The typical (median) train's own change is tiny (+6 s) — yet the *distribution* shifts **+29 s** at its median, because 54% of trains lose time and 45% gain, and the losses run almost twice the size of the gains. The junction disperses delay — both tails stretch — rather than adding a fixed amount to everyone.

What the shift looks like

The junction does change delay - by dispersing it, not shifting it



Early trains leave *earlier*; late trains leave *much* later. **71.3% of trains change by less than a minute either way** — the 24.8% that gain more than a minute account for essentially all the delay the junction adds. That concentration is why Questions 3–5 exist.

What is a chain, and do they really happen?

Definition. Train i links to train $i-1$ iff both were *moved* (gained >60 s) *and* their headway is ≤ 5 min, on the same track. A chain is a maximal run of linked trains.

Baseline / null hypothesis: if trains were **independent**, knowing that one was moved would tell you nothing about the next one — $P(\text{next moved} \mid \text{this moved}) = P(\text{next moved})$ unconditionally.

headway to next	$P(\text{next moved} \mid \text{this moved})$	$P(\text{next} \mid \text{this NOT})$	ratio
<3 min	92.1%	49.0%	1.9×
3–5 min	71.9%	14.6%	4.9×
5–10 min	28.5%	8.5%	3.4×
>10 min	14.0%	9.1%	1.5×

Answer: independence is firmly rejected. Of 89,463 moved events, **37.6% chain to at least one more**; 63.2% of all moved trains sit inside a chain of 2+; chains up to **18 trains** observed. Threshold checked from 15 s to 300 s: the chained share stays ≈ 58 –60% throughout — not an artifact of the 60 s cutoff.

Can we predict how much delay a train gains? (regression)

Baselines: zero (predict no change), persistence (exit delay = entry delay — the junction changes nothing), mean-gain (persistence plus the average gain).

GBDT features: own entry delay, hour, tunnel track, direction, service (base) + headway to leader, leader's entry delay (interaction block).

model	MAE	RMSE
zero	171.4 s	380.3 s
persistence	60.2 s	99.4 s
mean-gain	63.1 s	94.1 s
GBDT, no interaction features	66.1 s	143.2 s
GBDT, + headway/leader state	57.4 s	127.1 s

The no-interaction row loses to persistence on both columns.

Answer: marginal, and fragile. Full-year rolling origin (9 folds): mean +4.0% MAE over persistence, **wins in 8 of 9 months, loses in July**; RMSE is **worse in every single fold**. Regression is not the question this data answers well — see Q1: most of the target is 71% noise around zero.

Can we classify whether a train will be meaningfully delayed?

Target: gain >60 s (base rate 25.1%). **Baseline:** the trivial rule “headway <5 min $\Rightarrow$ at risk”.

GBDT features: headway to leader, leader’s entry delay, own entry delay, hour, month, tunnel track, direction, service, temperature, rain, snowfall.

model	ROC AUC	PR AUC	lift @ top 10%
trivial rule (headway <5 min)	0.692	0.371	1.71×
GBDT, all features	0.811	0.639	3.08×

At the stricter 120 s threshold: 0.703 $\rightarrow$ 0.828, lift 3.93×

Answer: a clear, stable win. Tested on 3 held-out months never seen in training. Calibration is honest too: predicted risk 0.04–0.81, actual 0.05–0.77. **Reformulating the target from “how many seconds” to “will it cross a threshold” is what makes the signal usable** — same features, same data, a much better-posed question.

Can we classify whether a train will trigger a domino?

Target: at the moment train i exits, will the *next* train on the same track also be moved?

Baseline: the naive rule “I’m late, so is whoever’s next”.

GBDT features: this train’s own moved status and exit delay, gap to the next train (actual or scheduled — see below), hour, month, tunnel track, direction, service, temperature, rain, snowfall.

model	ROC AUC	PR AUC	lift @ top 10%
naive: “I’m late, so is next”	0.689	0.397	—
retrospective GBDT (uses actual next-gap)	0.864	0.777	3.69×
operational GBDT (uses only the <i>scheduled</i> next-gap)	0.859	0.738	3.47×

Answer: the strongest predictive result in this deck, and the only one directly deployable as a real-time alert — the operational version needs only the timetable and the current train’s own state. Checked that this is not just autocorrelation: even a train that is currently *on time* still scores AUC 0.787 from headway and context alone.

Features used in each model

All models: gradient-boosted trees (`HistGradientBoostingClassifier` / `Regressor`), trained on months 1–9, tested on 10–12, held out. **Every feature is knowable at the prediction moment** — entry to the junction (Q3, Q4) or exit (Q5). No feature ever looks into the future.

	Q3 regression	Q4 classify moved	Q5 domino
own state	entry delay	entry delay	moved status, exit delay
neighbour context	headway, leader's entry delay hour, track, direction, service	headway, leader's entry delay hour, month, track, direction, service	gap to <i>next</i> train hour, month, track, direction, service
weather	—	temperature, rain, snowfall	temperature, rain, snowfall

Q5's “gap to next train” comes in two versions: **actual** (only knowable once that train has departed — retrospective) and **scheduled** (known from the timetable in advance — operational, the deployable one).

By permutation importance (Q4 model): **headway dominates** (AUC drop 0.172), then hour (0.062), track (0.020), leader's delay (0.019), own delay (0.013). Weather contributes ≈ 0 in every model tested.

Does delay skip a hop, or is propagation strictly local?

Setup. For chains of length ≥ 3 ($A \rightarrow B \rightarrow C$, same track, linked as in Q2): does A (two hops back) still predict C's exit delay once B's *full* state is controlled for?

Baseline / null hypothesis: if propagation is **strictly local** (pure relay — each train only reacts to whoever is immediately ahead), then controlling for B should fully explain away A's association with C. The partial correlation should collapse to ≈ 0 .

quantity	value
raw $r(\text{A's exit delay, C's exit delay})$	+0.359
partial $r \mid \text{B's exit/entry delay, both headways, hour, month}$	+0.228

$n = 8,836$ chains of length ≥ 3 .

Answer: no — the null is rejected. 63% of the raw association survives controlling for B. Not pure first-order relay: the chain's *starter* still matters two hops downstream, beyond what its immediate neighbor explains. That residual is what Question 7 goes looking for.

Is “station” — not just the train ahead — a common cause?

Setup. Two trains on *different* tracks cannot physically queue behind each other. Compare same-track vs. cross-track correlation of delay **at Central** (the identified bottleneck), **matched on time-gap** so proximity cannot confound the comparison.

Baseline / null hypothesis: if propagation is **purely track-specific** (relay/queueing only), cross-track correlation should collapse toward independence at every time-gap, since there is no physical mechanism linking two different tracks.

time gap	same-track r	n	cross-track r	n
60–120s	0.373	1,166	0.131	490,848
180–300s	0.419	289,060	0.124	962,672
450–600s	0.303	270,518	0.138	1,227,524

Answer: the null is rejected too — both mechanisms are real. Cross-track correlation is **flat at ≈ 0.13 across every distance** (station-wide ambient congestion, not decaying with proximity) — *but* same-track is **2–4 $\times$ higher** at every matched gap. Station congestion is a genuine common cause, and it explains part of Q6’s residual — but it does not explain it away: sharing a track still adds a large, independent effect on top.

The seven answers, together

	question	baseline	result
Q1	does the distribution change?	no-change null	yes — disperses
Q2	what is a chain?	independence	37.6% chain
Q3	predict seconds gained?	persistence	marginal, fragile
Q4	classify “meaningfully delayed”?	headway rule	AUC 0.81
Q5	classify “triggers a domino”?	naive propagation	AUC 0.86
Q6	does delay skip a hop?	full mediation	63% survives
Q7	is station a common cause?	pure track relay	both real

The pattern across all seven

The sharper and more concentrated the question, the more the model beats its baseline. **Regression on the whole population is weak** because most of the population barely moves (Q1). **Classification on the affected minority is strong** (Q4), and **classification of propagation is strongest of all** (Q5) — because that is where the causal mechanism lives. **Q6–Q7 open that mechanism up**: propagation is not a simple relay — it is a mix of local track queueing and ambient station-wide congestion, **both genuine, both quantified, neither explaining the other away**.